

Linux Kernel: Built-in Modules

TABLE OF CONTENTS

Introduction	2
Procedure	2
1. PetaLinux Setup and Project Preparation	2
2. Kernel Configuration	2
3. Adding Multiplier Driver to Kernel Source	3
4. Creating Build Configuration Files	3
5. Linking External Kernel Source	3
6. Enabling Built-in Driver	3
7. Kernel Build and Deployment	3
8. Verification of Built-in Driver	4
9. Kernel Optimization	4
Results	4
Conclusion	5
Knowledge Transfer	5
Appendix	6

Introduction

The objective of this laboratory exercise was to understand the integration of device drivers as **built-in modules within the Linux kernel** in an embedded Linux environment on the Zybo Z7-10 board using PetaLinux. Unlike the previous lab where the multiplier driver was implemented as a **loadable kernel module (LKM)**, this lab focuses on embedding the driver directly into the kernel so that it is automatically initialized during system boot.

This lab also introduces kernel configuration using **Kconfig and menuconfig**, allowing selective inclusion and exclusion of kernel components. Additionally, unnecessary drivers such as networking, multimedia, and sound support were disabled to generate a **reduced-size kernel image**, improving system efficiency.

This exercise provided practical exposure to kernel customization, built-in driver integration, and kernel size optimization.

Procedure

1. PetaLinux Setup and Project Preparation

1. A PetaLinux installation directory was created under /tmp for the user workspace:

```
cd /tmp  
mkdir petalinux_nithingowtham25
```
2. The PetaLinux installer was copied from the shared lab directory:

```
cd petalinux_nithingowtham25  
cp /mnt/lab_files/ECEN449/petalinux-v2023.1-05012318-installer.run .
```
3. A directory was created for installation and the installer was executed:

```
mkdir petalinux_install  
chmod 755 ./petalinux-v2023.1-05012318-installer.run  
./petalinux-v2023.1-05012318-installer.run --dir petalinux_install --platform "arm"
```
4. After installation, environment variables were loaded:

```
source petalinux_install/settings.sh
```
5. A new PetaLinux project was created using the Zynq template:

```
petalinux-create -t project --template zynq -n linux_boot_nithingowtham25 --tmpdir  
/tmp/petalinux_nithingowtham25/tmp_dir
```
6. The hardware configuration was updated using the exported Vivado design:

```
cd linux_boot_nithingowtham25  
petalinux-config --get-hw-  
description=/home/grads/n/nithingowtham25/Spring2026/ECEN749/lab4_new/linux_boot_wrapper.x  
sa --silentconfig
```

2. Kernel Configuration

7. The kernel configuration menu was accessed using:

```
petalinux-config -c kernel
```

8. The menuconfig interface was explored to understand kernel options and dependencies.

3. Adding Multiplier Driver to Kernel Source

9. The Linux source code was extracted into the PetaLinux project directory under:

components/ext_sources/

10. A new directory was created for the driver: drivers/multiplier_driver/

11. The multiplier driver source files (multiplier.c and headers) were copied into this directory (shown in [Appendix I](#)).

4. Creating Build Configuration Files

12. A **Makefile** was created in the driver directory with the following entry:

obj-\$(CONFIG_MULTIPLIER_DRIVER) += multiplier.o

13. A **Kconfig** file was created with:

- Tristate configuration (Y/M/N)
- Dependency on ARM architecture
- Default enabled for ARM systems

14. The driver was linked to the kernel build system:

- Added entry in drivers/Makefile
- Added source in drivers/Kconfig

5. Linking External Kernel Source

15. The modified Linux source was linked to the PetaLinux project:

petalinux-config

16. External kernel source path was provided under:

Linux Components Selection → linux-kernel → ext-local-src

6. Enabling Built-in Driver

17. Kernel configuration was reopened:

petalinux-config -c kernel

18. The multiplier driver was visible under the **Device Drivers** menu in menuconfig, confirming successful integration into the kernel configuration (shown in [Appendix A](#)).

19. The multiplier driver was enabled as: Device Drivers → multiplier driver → Built-in (Y)

7. Kernel Build and Deployment

20. Kernel version check was skipped by modifying the .bb file.

21. The system was built:

petalinux-build

(Build success shown in [Appendix B](#))

22. The generated image.ub size was observed as **18 MB** (shown in [Appendix C](#)).

23. The generated image.ub was copied to the SD card.

24. The system was booted on the Zybo Z7-10 board, and the boot sequence was verified (shown in [Appendix D](#)).

8. Verification of Built-in Driver

25. During boot, the initialization message from the multiplier driver was observed ([Appendix D](#)).

26. Since the driver is built into the kernel, no insmod command was required.

27. The device node was created using:

```
mknod /dev/multiplier c 244 0
```

28. The user application (devtest) was executed:

```
./devtest
```

Execution shown in [Appendix E](#)

29. Correct multiplication results were verified using devtest output (shown in [Appendix F](#)).

9. Kernel Optimization

30. The following drivers were disabled using menuconfig (shown in [Appendix G](#)):

- Network device support
- Multimedia support
- Sound card support

31. The kernel was rebuilt: petalinux-build

32. The reduced kernel image size of **16.5 MB** was observed and compared with the initial size (shown in [Appendix H](#)).

(The complete working of the kernel was demonstrated to the TA)

Results

The embedded Linux system successfully booted with the multiplier driver integrated as a **built-in kernel module**, as shown in [Appendix D](#). The driver initialization message appeared during boot, confirming that the driver was automatically loaded by the kernel.

The kernel build was completed successfully ([Appendix B](#)), and the initial kernel image size (**18MB**) was recorded ([Appendix C](#)). After disabling unnecessary drivers, the kernel was rebuilt, resulting in a significantly reduced image size of **16.5MB** ([Appendix H](#)).

The user application executed successfully ([Appendix E](#)), and the devtest output ([Appendix F](#)) confirmed correct multiplication functionality. The results matched expected values for all tested inputs.

The following key observations were made:

- The driver was automatically initialized during boot without manual insertion
- Kernel configuration successfully controlled inclusion of drivers
- System functionality remained intact after removing unused drivers
- Kernel image size was significantly reduced after optimization
- Built-in modules improved boot-time readiness of the driver

The successful operation of the built-in driver and optimized kernel was demonstrated to the TA.

Conclusion

This laboratory exercise successfully demonstrated the integration of a device driver as a **built-in** component of the Linux kernel. By modifying the kernel source, configuring Kconfig entries, and enabling the driver through menuconfig, a deeper understanding of kernel-level design and driver integration was achieved.

The multiplier driver was automatically initialized during system boot, eliminating the need for manual insertion and improving system readiness. This highlights the advantage of built-in modules in embedded systems where reliability and minimal user intervention are important.

Additionally, **kernel optimization** was performed by disabling unnecessary drivers such as network, multimedia, and sound support. This resulted in a noticeable **reduction in kernel image size**, demonstrating the importance of customizing the kernel based on system requirements.

Overall, this lab reinforced concepts of kernel configuration, built-in driver integration, and system optimization, while also emphasizing the trade-offs between flexibility and efficiency in embedded Linux systems.

Knowledge Transfer

a) What are the advantages and disadvantages of loadable kernel modules and built-in modules?

BUILT-IN MODULES

Advantages:

- Automatically loaded during system boot
- No need for manual insertion (insmod)
- Faster availability during runtime

Disadvantages:

- Increases kernel size
- Cannot be removed or modified without recompiling the kernel
- Less flexible compared to modular approach

LOADABLE KERNEL MODULES (LKM)

Advantages:

- Can be dynamically loaded and unloaded
- Easier debugging and updates without rebuilding kernel
- Keeps base kernel smaller and more modular

Disadvantages:

- Requires manual loading
- Slight runtime overhead due to dynamic insertion
- Dependency management required between modules

Appendix

APPENDIX A – Multiplier Driver in Kernel Config

The screenshot shows a terminal window with the 'linux-xlnx Configuration' dialog box open. The dialog is titled '.config - Linux/arm 6.1.70 Kernel Configuration'. It displays a list of device drivers with checkboxes. The 'multiplier driver' is highlighted and selected. The background shows a terminal with various status messages and a file explorer window.

```

NOTE: Tasks Summary: Attempted 4135 tasks of which 972 didn't need to be rerun and all succeeded.

Summary
INFO: F
[INFO]
nithing
ux-conf
[INFO]
[INFO]
[INFO]
[INFO]
[INFO]
[INFO]
NOTE: S
d/cache
Loading
Loaded
Parsing
Parsing
errors
NOTE: R
Initial
Sstate
NOTE: N
NOTE: E
NOTE: T
[INFO]
NOTE: S
d/cache

```

linux-xlnx Configuration

.config - Linux/arm 6.1.70 Kernel Configuration

> Device Drivers

Device Drivers

Arrow keys navigate the menu. <Enter> selects submenus --- (or empty submenus ---). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in []

< > FSI support ---

< > Trusted Execution Environment support ---

< > Eckelmann SIOX Support ---

< > SLIMbus support ---

[] On-Chip Interconnect management support ---

< > Counter support ---

< > MOST support ---

< > PECI support ---

[] Hardware Timestamping Engine (HTE) Support ---

<*> multiplier driver

<select> < Exit > < Help > < Save > < Load >

APPENDIX B – Initial Build Success

The screenshot shows a terminal window with the following output:

```

nithingowtham25@zach-333-em4-06:/tmp/petalinux_nithingowtham25/linux_boot_...

Setscene tasks: 1448 of 1541
Setscene tasks: 1448 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Currently 22 running tasks (4 of 4135) 0% |
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
Setscene tasks: 1541 of 1541
NOTE: linux-xlnx: compiling from external source tree /tmp/petalinux_nithingowtham25/linux_boot_ni
thingowtham25/components/ext_sources/linux-xlnx-xlnx_rebase_v6.1_LTS
NOTE: Tasks Summary: Attempted 4135 tasks of which 972 didn't need to be rerun and all succeeded.

Summary: There was 1 WARNING message.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
nithingowtham25@zach-333-em4-06:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25$

```

APPENDIX C – Initial Image Size

The screenshot shows a file explorer window with the following files and their sizes:

Name	Size	Modified
pxelinux.cfg	—	11:28
boot.scr	3.0 kB	11:28
config	7.1 kB	11:26
image.ub	18.0 MB	11:51
rootfs.cpio	34.6 MB	11:51
rootfs.cpio.gz	13.2 MB	11:51
rootfs.cpio.gz.u-boot	13.2 MB	11:51

APPENDIX D – Boot Initialization Message

```
nithingowtham25@zach-333-em4-06:/tmp/petalinux_nithingowtham25/linux_boot_...
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x
EDAC MC: ECC not enabled
Xilinx Zynq CpuIdle Driver started
sdhci: Secure Digital Host Controller Interface driver
sdhci: Copyright(c) Pierre Ossman
sdhci-pltfm: SDHCI platform and OF driver helper
ledtrig-cpu: registered to indicate activity on CPUs
clocksource: ttc_clocksource: mask: 0xffff max_cycles: 0xffff, max_idle_ns: 537538477 ns
timer #0 at (ptrval), irq=41
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
fpga_manager fpga0: Xilinx Zynq FPGA Manager registered
Mapping virtual address...
Physical address = 0x43c00000
Virtual address = (ptrval)
Registered a device with dynamic Major number of 244
Create a device file for this device with this command:
'mknod /dev/multiplier c 244 0'.
mmc0: SDHCI controller on e0100000.mmc [e0100000.mmc] using ADMA
NET: Registered PF_INET6 protocol family
Segment Routing with IPv6
In-situ OAM (IOAM) with IPv6
sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
NET: Registered PF_PACKET protocol family
can: controller area network core
NET: Registered PF_CAN protocol family
can: raw protocol
can: broadcast manager protocol
can: netlink gateway - max_hops=1
Registering SWP/SWPB emulation handler
of-fpga-region fpga-full: FPGA Region probed
of_cfs_init
of_cfs_init: OK
ALSA device list:
No soundcards found.
mmc0: new high speed SDHC card at address 0001
mmcblk0: mmc0:0001 MS 7.32 GiB
mmcblk0: p1
Freeing initrd memory: 12856K
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process
INIT: version 3.04 booting
Starting udev
Starting version 251.8+
```

APPENDIX E – User Application Running

```
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x
linux_boot_nithingowtham25:~$ sudo su
We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

Password:
linux_boot_nithingowtham25:/home/petalinux# mount /dev/mmcblk0p1 /mnt/
/dev/mmcblk0p1: Can't open blockdev
/dev/mmcblk0p1: Can't open blockdev
/dev/mmcblk0p1: Can't open blockdev
linux_boot_nithingowtham25:/home/petalinux# cd /mnt/
linux_boot_nithingowtham25:/mnt# ./devtest
Failed to open device file!
linux_boot_nithingowtham25:/mnt# cd ..
linux_boot_nithingowtham25:/# mknod /dev/multiplier c
BusyBox v1.35.0 () multi-call binary.

Usage: mknod [-m MODE] NAME TYPE [MAJOR MINOR]

Create a special file (block, character, or pipe)

-m MODE Creation mode (default a=rw)
TYPE:
b      Block device
c or u Character device
p      Named pipe (MAJOR MINOR must be omitted)
linux_boot_nithingowtham25:/# mknod /dev/multiplier c 244 0
linux_boot_nithingowtham25:/# cd /mnt/
linux_boot_nithingowtham25:/mnt# ./devtest
The multiplier device is opened successfully!!!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 0
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 0
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
```

APPENDIX F – devtest output matched

```
nithingowtham25@zach-333-em4-06:/tmp/petalinux_nithingowtham25/linux_boot_...
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e...
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
Multiplier: Read byte[7] = 0
Multiplier: Read byte[8] = 208
Multiplier: Read byte[9] = 0
Multiplier: Read byte[10] = 0
Multiplier: Read byte[11] = 0
Multiplier: Total bytes read = 12
16 * 13 = 208 Result Correct!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 16
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 14
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
Multiplier: Total bytes written = 8
Multiplier: Read requested = 12 bytes, serving = 12 bytes
Multiplier: Read byte[0] = 16
Multiplier: Read byte[1] = 0
Multiplier: Read byte[2] = 0
Multiplier: Read byte[3] = 0
Multiplier: Read byte[4] = 14
Multiplier: Read byte[5] = 0
Multiplier: Read byte[6] = 0
Multiplier: Read byte[7] = 0
Multiplier: Read byte[8] = 224
Multiplier: Read byte[9] = 0
Multiplier: Read byte[10] = 0
Multiplier: Read byte[11] = 0
Multiplier: Total bytes read = 12
16 * 14 = 224 Result Correct!
Multiplier: Write requested = 8 bytes, writing = 8 bytes
Multiplier: Wrote byte[0] = 16
Multiplier: Wrote byte[1] = 0
Multiplier: Wrote byte[2] = 0
Multiplier: Wrote byte[3] = 0
Multiplier: Wrote byte[4] = 15
Multiplier: Wrote byte[5] = 0
Multiplier: Wrote byte[6] = 0
Multiplier: Wrote byte[7] = 0
```

APPENDIX G – Excluded Drivers

```
nithingowtham25@zach-333-em4-06:/tmp/petalinux_nithingowtham25/linux_boot_...
nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e... x nithingowtham25@zach-333-e...
NOTE: Tasks Summary: Attempted 4135 tasks of which 972 didn't need to be rerun and all succeeded.

Summary
INFO: F
[INFO]
nithing
ux-conf
[INFO]
[INFO]
[INFO]
[INFO]
[INFO]
NOTE: S
d/cache
Loading
Loaded
Parsing
errors
NOTE: R
Initial
State
NOTE: M
NOTE: E
NOTE: T
NOTE: S
d/cache
Loading
Loaded
Parsing
errors
NOTE: R
Initial
State
NOTE: E
Setscen
Current
by: linu

linux-xlnx Configuration
.config - linux/arm 6.1.70 Kernel Configuration
Device Drivers
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty
submenus ---). Highlighted letters are hotkeys. Pressing <Y>
includes, <N> excludes, <M> modularizes features. Press <Esc>=Esc to
exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
[?]
[*]
IEEE 1394 (FireWire) support ---
[*] Network device support ---
Input device support ---
Character devices ---
I2C support ---
< > I3C support ---
[*] SPI support ---
< > SPI support ---
< > HSI support ---
< > PPS support ---
PTP clock support ---
* Pln controllers ---
* GPIO support ---
< > Dallas's i-wire support ---
* Board Level reset or power off ---
* Power supply class support ---
< > Hardware Monitoring support ---
[*] Thermal drivers ---
[*] Watchdog timer support ---
< > Sonics Silicon Backplane support ---
< > Broadcom specific AMBA ---
Multifunction device drivers ---
[*] Voltage and Current Regulator support ---
< > Remote Controller support ---
CEC support ---
< > Multimedia support ---
Graphics support ---
< > Sound card support ---
HID support ---
[*] USB support ---
V(?)
[select] < Exit > < Help > < Save > < Load >
```


APPENDIX H – Reduced Image Size

Name	Size	Modified
pxelinux.cfg	1 item	11:28
boot.scr	3.0 kB	11:28
config	7.1 kB	12:17
image.ub	16.5 MB	12:19

APPENDIX I (Source Code)

• multiplier.h

```
6-character-device-driver > multiplier > C multiplier.h > init_module(void)
1  /* All of our linux kernel includes. */
2  #include <linux/module.h> /* Needed by all modules */
3  #include <linux/moduleparam.h> /* Needed for module parameters */
4  #include <linux/kernel.h> /* Needed for printk and KERN_* */
5  #include <linux/init.h> /* Need for __init macros */
6  #include <linux/io.h>
7  #include <asm/io.h> /* Needed for IO reads and writes */
8  #include <linux/ioport.h>
9  #include <linux/slab.h>
10 #include <linux/fs.h> /* Provides file ops structure */
11 #include <linux/sched.h> /* Provides access to the "current" process
12 | task structure */
13 #include <asm/uaccess.h> /* Provides utilities to bring user space
14 | data into kernel space. Note, it is
15 | processor arch specific. */
16
17
18 /* Some defines */
19 #define DEVICE_NAME "multiplier"
20 #define BUF_LEN 80
21
22 /* Function prototypes, so we can setup the function pointers for dev
23 | file access correctly. */
24 int init_module(void);
25 void my_exit(void);
26 static int device_open(struct inode *, struct file *);
27 static int device_close(struct inode *, struct file *);
28 static ssize_t device_read(struct file *, char *, size_t, loff_t *);
29 static ssize_t device_write(struct file *, const char *, size_t, loff_t *);
30
31 /*
32 | * Global variables are declared as static, so are global but only
33 | * accessible within the file.
34 | */
35 static int Major; /* Major number assigned to our device driver */
36 static int Device_Open = 0; /* Flag to signify open device */
```

• multiplier.c

```
6-character-device-driver > multiplier > C multiplier.c > ...
11 /* Moved all prototypes and includes into the headerfile */
12 #include "multiplier.h"
13 #include "xparameters.h" /* needed for physical address of multiplier */
14
15 /* from xparameters.h */
16 #define PHY_ADDR XPAR_MULTIPLY_0_S00_AXI_BASEADDR /* physical address of multiplier */
17
18 /* size of physical address range for multiply */
19 #define MEMSIZE (XPAR_MULTIPLY_0_S00_AXI_HIGHADDR - XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1)
20
21 void *virt_addr; /* virtual address pointing to multiplier */
22
23 /* This structure defines the function pointers to our functions for
24 | opening, closing, reading and writing the device file. There are
25 | lots of other pointers in this structure which we are not using,
26 | see the whole definition in linux/fs.h */
27 static struct file_operations fops = {
28     .read = device_read,
29     .write = device_write,
30     .open = device_open,
31     .release = device_close
32 };
```

```

44 int my_init(void)
45 {
46
47     /* Linux kernel's version of printf */
48     printk(KERN_INFO "Mapping virtual address...\n");
49
50     /* map virtual address to multiplier physical address */
51     /* use ioremap */
52     virt_addr = ioremap(PHY_ADDR, MEMSIZE);
53
54     if (!virt_addr) {
55         printk(KERN_ALERT "ioremap failed!\n");
56         return -ENOMEM;
57     }
58
59     /* print physical and virtual addresses */
60     printk(KERN_INFO "Physical address = %x\n", PHY_ADDR);
61     printk(KERN_INFO "Virtual address = %p\n", virt_addr);
62
63
64     /* This function call registers a device and returns a major number
65      * associated with it. Be wary, the device file could be accessed
66      * as soon as you register it, make sure anything you need (ie
67      * buffers ect) are setup BEFORE you register the device.*/
68     Major = register_chrdev(0, DEVICE_NAME, &fops);
69
70
71     /* Negative values indicate a problem */
72     if (Major < 0) {
73         /* Make sure you release any other resources you've already
74          * grabbed if you get here so you don't leave the kernel in a
75          * broken state. */
76         printk(KERN_ALERT "Registering char device failed with %d\n", Major);
77         return Major;
78     }
79
80     printk(KERN_INFO "Registered a device with dynamic Major number of %d\n", Major);
81
82     printk(KERN_INFO "Create a device file for this device with this command:\nmknod /dev/%s c %d 0'\n", DEVICE_NAME, Major);

```

```

84     return 0; /* success */
85 }
86
87 /*
88  * This function is called when the module is unloaded, it releases
89  * the device file.
90  */
91 void my_exit(void)
92 {
93     /* Unregister the device */
94     unregister_chrdev(Major, DEVICE_NAME);
95
96     /* Unmap virtual memory */
97     printk(KERN_ALERT "unmapping virtual address space....\n");
98     iounmap((void *)virt_addr);
99
100 }

```

```

107 static int device_open(struct inode *inode, struct file *file)
108 {
109
110     /* In these case we are only allowing one process to hold the device
111      * file open at a time. */
112     if (Device_Open) /* Device_Open is my flag for the usage of the device
113                      * file (defined in multiplier.h) */
114         return -EBUSY; /* Failure to open device is given
115                      * back to the userland program. */
116     Device_Open++; /* Keeping the count of the device opens. */
117
118
119     try_module_get(THIS_MODULE); /* increment the module use count
120                                  * (make sure this is accurate or you
121                                  * won't be able to remove the module
122                                  * later. */
123
124     /* Print kernel message */
125     printk(KERN_INFO "The multiplier device is opened successfully!!!\n");
126     return 0;
127 }
128
129 /*
130  * Called when a process closes the device file.
131  */
132 static int device_close(struct inode *inode, struct file *file)
133 {
134     Device_Open--; /* We're now ready for our next caller */
135
136     /*
137      * Decrement the usage count, or else once you opened the file,
138      * you'll never get rid of the module.
139      */
140     module_put(THIS_MODULE);
141
142     /* Print kernel message */
143     printk(KERN_INFO "The multiplier device is closed successfully!!!\n");
144     return 0;
145 }

```

```

151 static ssize_t device_read(struct file *filp, /* see include/linux/fs.h */
152                            char __user *buffer, /* buffer to fill with data */
153                            size_t length, /* length of the buffer */
154                            loff_t * offset)
155 {
156     int i;
157     int bytes_read = 0;
158
159     /*
160      * The user may request any number of bytes, but the multiplier
161      * peripheral only supports addresses from byte 0 to byte 11.
162      * So we compute a safe length that does not exceed 12 bytes.
163      */
164     size_t valid_len = (length <= 12) ? length : 12;
165
166     /* Print how many bytes were requested vs how many will be served */
167     printk(KERN_INFO "Multiplier: Read requested = %zu bytes, serving = %zu bytes\n",
168            length, valid_len);
169
170     /*
171      * Loop through each valid byte in the peripheral address space.
172      * For each byte:
173      * - Read from the hardware using ioread32()
174      * - Copy it to user space using put_user()
175      */
176     for (i = 0; i < valid_len; i++) {
177
178         /* Read one byte from the memory-mapped peripheral */
179         char data = ioread32(virt_addr + i);
180
181         /*
182          * Copy the byte from kernel space to user space buffer.
183          * 'buffer' is a user-space pointer, so put_user is required.
184          */
185         put_user(data, buffer + i);
186
187         /* Increment count of successfully transferred bytes */
188         bytes_read++;
189
190         printk(KERN_INFO "Multiplier: Read byte[%d] = %d\n", i, data);
191     }
192
193     /*
194      * Return the number of bytes actually transferred to user space.
195      * This satisfies the requirement of the read() system call.
196      */
197     printk(KERN_INFO "Multiplier: Total bytes read = %d\n", bytes_read);
198     return bytes_read;
199 }
200
201 static ssize_t device_write(struct file *file, const char __user * buffer, size_t length, loff_t * offset)
202 {
203     int i;
204     int bytes_written = 0;
205
206     /*
207      * The multiplier peripheral only supports writes to byte
208      * addresses 0 through 7. So we compute a safe length that
209      * does not exceed 8 bytes.
210      */
211     size_t valid_len = (length <= 8) ? length : 8;
212
213     /* Print how many bytes were requested vs how many will be written */
214     printk(KERN_INFO "Multiplier: Write requested = %zu bytes, writing = %zu bytes\n",
215            length, valid_len);
216
217     /*
218      * Loop through each byte to be written:
219      * - Copy data from user space to kernel variable using get_user()
220      * - Write that byte to the memory-mapped peripheral using iowrite32()
221      */
222     for (i = 0; i < valid_len; i++) {
223
224         char data;
225
226         /*
227          * Copy one byte from user space buffer to kernel space.
228          * 'buffer' is a user-space pointer, so get_user is required.
229          */
230         get_user(data, buffer + i);
231
232         /*
233          * Write the byte to the corresponding offset in the
234          * multiplier peripheral address space.
235          */
236         iowrite32(data, virt_addr + i);
237
238         /* Increment count of successfully written bytes */
239         bytes_written++;
240
241         /* Debug print for each byte written */
242         printk(KERN_INFO "Multiplier: Wrote byte[%d] = %d\n", i, data);
243     }
244
245     /*
246      * Return the number of bytes successfully written to the device.
247      * This satisfies the requirement of the write() system call.
248      */
249     printk(KERN_INFO "Multiplier: Total bytes written = %d\n", bytes_written);
250     return bytes_written;
251 }
252
253 /* These define info that can be displayed by modinfo */
254 MODULE_LICENSE("GPL");
255 MODULE_AUTHOR("Nithin Gowtham Saravanan");
256 MODULE_DESCRIPTION("Module which creates a multiplier character device and allows user interaction with it");
257
258 /* Here we define which functions we want to use for initialization
259    and cleanup */
260 module_init(my_init);
261 module_exit(my_exit);
262
263

```